4.4 Deployment i Monitoring: Wdrażanie na AWS Lambda (serverless) oraz monitorowanie kosztów i jakości odpowiedzi AI

Moduł 4: Szybkie Wdrażanie (Ship Fast) w oparciu o Angular

Krzysztof Rytlewski · 2026

Architektura serverless — czym jest AWS Lambda?

AWS Lambda to usługa obliczeniowa działająca w modelu **Function as a Service (FaaS)**. Zamiast utrzymywać własny serwer, piszesz funkcję uruchamianą na żądanie — AWS automatycznie zarządza infrastrukturą, skalowaniem i dostępnością.

Model rozliczenia: płacisz wyłącznie za czas wykonania (z dokładnością do milisekund) i liczbę wywołań. Przy umiarkowanym ruchu (do ~1 miliona wywołań miesięcznie) Lambda mieści się w bezpłatnym tierze AWS.

Lambda jest idealna dla backendu AI: wywołania są sporadyczne, zimny start Node.js wynosi ~200–500 ms (akceptowalny dla czatbota), a automatyczne skalowanie obsługuje jednocześnie tysiące użytkowników.

Architektura wdrożenia aplikacji AI

```
[Przeglądarka / Angular SPA]
        |  HTTPS
        ▼
[AWS CloudFront] ──────────────────► [S3 bucket]
        |                             (Angular build)
        |  /api/*
        ▼
[AWS API Gateway]
        |
        ▼
[AWS Lambda] (Node.js runtime)
        |
        ├──► [OpenAI / Anthropic API]
        └──► [MongoDB Atlas]
```

CloudFront serwuje statyczny bundle Angular z S3, a żądania `/api/*` przekierowuje do API Gateway, który wywołuje Lambda z backendem Node.js.

Przygotowanie Express do wdrożenia jako Lambda

Lambda oczekuje eksportowanej funkcji `handler`. Biblioteka `@codegenie/serverless-express` pozwala przenieść istniejącą aplikację Express na Lambda przy minimalnej zmianie kodu.

```
// lambda.js — punkt wejścia dla AWS Lambda
import serverlessExpress from '@codegenie/serverless-express';
import app from './app.js';   // istniejąca aplikacja Express

export const handler = serverlessExpress({ app });
```

Plik `app.js` zawiera konfigurację Express **bez** wywołania `app.listen()` — nasłuchiwanie obsługuje Lambda. Lambda reużywa kontenera między wywołaniami (*warm instance*) — połączenie z MongoDB nawiązane przy pierwszym wywołaniu pozostaje otwarte dla kolejnych.

CI/CD Pipeline — GitHub Actions dla Lambda i S3

Automatyczne wdrożenie przy każdym `push` do `main` gwarantuje spójność między kodem w repozytorium a produkcją. Pipeline składa się z dwóch niezależnych jobów:

Job `deploy-backend`: instaluje zależności produkcyjne (`npm ci --omit=dev`), tworzy archiwum ZIP i aktualizuje kod funkcji Lambda przez AWS CLI.

Job `deploy-frontend`: instaluje zależności, buduje Angular produkcyjnie (`ng build --configuration production`), synchronizuje pliki do S3 z flagą `--delete` i unieważnia cache CloudFront.

Sekrety (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_BUCKET`, `CF_DISTRIBUTION_ID`) definiujemy w ustawieniach repozytorium GitHub — **nigdy** bezpośrednio w pliku YAML.

Zmienne środowiskowe i sekrety w Lambda

Lambda udostępnia zmienne środowiskowe konfigurowane przez konsolę AWS lub CLI. Wrażliwe dane najlepiej przechowywać w AWS Secrets Manager.

Bezpieczna alternatywa — cachowanie sekretów w pamięci Lambda:

```
// secrets.js
import { SecretsManagerClient, GetSecretValueCommand } from '@aws-sdk/client-secrets-manager';

const client = new SecretsManagerClient({ region: 'eu-west-1' });
let cachedSecrets;

export async function getSecrets() {
  if (cachedSecrets) return cachedSecrets;
  const response = await client.send(
    new GetSecretValueCommand({ SecretId: 'ai-app/production' })
  );
  cachedSecrets = JSON.parse(response.SecretString);
```

Cachowanie w zmiennej modułu jest kluczowe — wywołanie Secrets Manager kosztuje i wydłuża zimny start.

Monitorowanie kosztów wywołań AI

Koszty aplikacji AI mogą rosnąć nieoczekiwanie szybko. Każda odpowiedź OpenAI zawiera metadane zużycia tokenów — zapisujemy je do MongoDB:

```
const { prompt_tokens, completion_tokens, total_tokens } = completion.usage;

await db.collection('usage_logs').insertOne({
  sessionId, userId: req.userId, model: 'gpt-4o',
  promptTokens: prompt_tokens,
  completionTokens: completion_tokens,
  // Szacowany koszt wg cennika OpenAI (kwiecień 2026)
  estimatedCostUsd: (prompt_tokens * 0.0000025) + (completion_tokens * 0.00001),
  ts: new Date(),
});
```

Dzienną agregację kosztów uzyskujemy przez MongoDB `group‘z‘dateToString` i `$sum` — gotowe dane do dashboardu kosztów.

Narzędzia monitorowania kosztów

Poza własną kolekcją `usage_logs` w MongoDB stosujemy trzy zewnętrzne narzędzia:

- **AWS Cost Explorer** — wizualizacja kosztów Lambda, API Gateway, S3 i CloudFront z podziałem na usługi i okresy czasu.

- **AWS Budgets** — automatyczne alerty email/SMS po przekroczeniu zdefiniowanego progu (np. $50 miesięcznie). Konfiguracja przez konsolę AWS — niezbędna od pierwszego dnia produkcji.

- **OpenAI Usage Dashboard** — oficjalny panel zużycia tokenów i kosztów po stronie dostawcy; niezależny od własnych logów.

Monitorowanie kosztów od pierwszego dnia produkcji zapobiega nieprzyjemnym niespodziankom na rachunku.

Strukturalne logowanie w Lambda

Lambda integruje się natywnie z **AWS CloudWatch** — wszystkie logi z `console.log()` trafiają automatycznie bez żadnej konfiguracji.

```
// logger.js
export function log(event, data = {}) {
  console.log(JSON.stringify({
    event, ...data,
    ts: new Date().toISOString(),
    requestId: process.env.AWS_REQUEST_ID,
  }));
}

// Użycie:
log('chat_request', { userId: req.userId, model: 'gpt-4o', tokens: total_tokens });
log('chat_error',   { userId: req.userId, error: err.message });
```

Strukturalne logi w formacie JSON umożliwiają łatwe filtrowanie i tworzenie metryk w CloudWatch Logs Insights.

Alarmy CloudWatch — wykrywanie problemów zanim zgłosi je użytkownik

CloudWatch Logs Insights pozwala filtrować logi przez SQL-podobny język zapytań — np. wszystkie błędy z ostatnich 24 godzin.

Alarm CloudWatch wysyła powiadomienie SNS, gdy metryka przekroczy próg:

```
aws cloudwatch put-metric-alarm \
  --alarm-name "LambdaErrorsHigh" \
  --metric-name Errors \
  --namespace AWS/Lambda \
  --statistic Sum --period 60 \
  --threshold 5 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:eu-west-1:123456789:AlertsTopic
```

Alarm reaguje na więcej niż 5 błędów Lambda w ciągu minuty i wysyła email przez SNS — inżynier dowiaduje się o problemie zanim zgłosi go użytkownik.

Podsumowanie — kompletny stack wdrożeniowy

Kompletna architektura „Ship Fast" łączy wszystkie warstwy w spójny system:

1. **Frontend** — Angular budowany przez GitHub Actions, wgrywany na S3, serwowany przez CloudFront z CDN.
2. **Backend** — Node.js/Express jako AWS Lambda z adapterem `serverless-express`, wyzwalany przez API Gateway.
3. **Baza danych** — MongoDB Atlas z indeksami zapewniającymi wydajność zapytań.
4. **Sekrety** — AWS Secrets Manager zamiast zmiennych środowiskowych wprost w konfiguracji.
5. **Monitoring** — CloudWatch Logs, AWS Budgets, własna kolekcja `usage_logs` w MongoDB.

Całość wdrażana automatycznie przy każdym `push` do `main` przez GitHub Actions CI/CD.